

SecureChat Report - Roll Number: Messam Raza - A02

Assignment 2 — SecureChat

Student: Messam Raza

Course: Information Security — Fall 2025

Github: <https://github.com/messam-raza/securechat-skeleton.git>

This report fully explains all implementation phases of the SecureChat system including PKI setup, certificate validation, Diffie–Hellman key exchange, AES encryption, signatures, replay protection, transcript logging, and session receipts.

1. Executive Summary

The SecureChat system implements a custom secure communication protocol combining:

- *X.509 certificates with a custom Certificate Authority (CA)*
- *Mutual certificate verification between client and server*
- *Ephemeral Diffie–Hellman key exchange with certificate-bound key derivation*
- *AES-128-CBC encryption with PKCS#7 padding*
- *RSA signatures for message integrity and sender authenticity*
- *Strict replay protection via monotonic sequence numbers*
- *Per-session transcript generation and signed SessionReceipt for non-repudiation*

The protocol is NOT TLS; it is a user-space secure channel constructed manually for learning purposes.

2. Project Structure & Workflow

Project folders modular architecture:

app/

crypto/

aes_util.py (AES encryption/decryption)

dh.py (Diffie-Hellman operations)

pki.py (certificate loader & validator)

sign.py (RSA signatures)

common/

protocol.py (Pydantic message formats)

utils.py (helpers such as base64, timestamps)

storage/

transcript.py (Transcript logging)

db.sql (MySQL schema: users table)

client.py

server.py

scripts/

gen_ca.py

gen_cert.py

verify_cert_chain.py

Execution:

- *Server starts → sends ServerHello*
- *Client verifies cert → sends Hello*
- *Ephemeral DH → derive temporary AES key*

- *Client registers or logs in (encrypted)*
- *Session DH → derive session AES key*
- *Chat messages encrypted + signed*
- *Close message generates transcript + receipt*

3. PKI Setup & Certificate Verification

The system uses:

- *CA certificate: ca.cert.pem*
- *Server certificate: server.cert.pem (CN=server)*
- *Client certificate: client.cert.pem (CN=client)*

Verification workflow:

- 1. Server sends ServerHello with its PEM certificate.*
- 2. Client loads CA certificate and verifies:*
 - *Signature*
 - *Validity (not expired)*
 - *SAN (hostname / IP matches)*
- 3. Client sends Hello including client certificate.*
- 4. Server validates client certificate similarly.*

Your verify_cert_against_ca() in pki.py performs:

- *Signature validation using CA public key*
- *CN comparison (expected_cn=client for client cert)*
- *SAN check for hostname matching*
- *Date validity check*

This prevents MITM using untrusted certificates.

4. Diffie–Hellman & Ephemeral AES Key Derivation

Your implementation uses RFC 3526 MODP Group 14 (2048-bit prime).

Steps:

- 1. Client sends $DHClient(p, g, A)$*
- 2. Server responds with $DHServer(B)$*
- 3. Both compute shared secret $Ks = g^{(ab)} \bmod p$*
- 4. Then derive AES key:*

$base_key = SHA256(Ks_big_endian)[:16]$

- 5. You strengthen security by binding key to certificate fingerprints:*

$aes_key = SHA256(base_key || client_fp || server_fp)[:16]$

This prevents unknown-key-share and man-in-the-middle attacks even if DH is replaced.

5. AES-128-CBC Encryption (Confidentiality)

Your aes_util.py implements:

- *pad_pkcs7()*
- *unpad_pkcs7()*
- *aes_encrypt()*
- *aes_decrypt()*

AES-128-CBC format:

iv | ciphertext

IV is random per message:

iv = os.urandom(16)

Both IV and ciphertext are base64-encoded and placed inside JSON messages.

6. Registration & Login (Phase 3)

Encrypted under ephemeral AES key.

Client sends:

```
{  
  "type": "register",  
  "iv": "...",  
  "ct": "..."  
}
```

Server decrypts → extracts JSON → applies salted hashing:

$\text{pwd_hash} = \text{SHA256}(\text{salt} || \text{password})$

SQL stored fields:

email | username | salt | pwd_hash

Login repeats same decryption and recomputes hash for validation.

7. Session Diffie–Hellman (Phase 4)

After successful login:

- *Server sends $\text{SessionDHServer}(p, g, B)$*
- *Client responds $\text{SessionDHClient}(A)$*
- *Both derive session_key*

This session key is used for all chat messages.

8. Chat Messages (Phase 6)

ChatMessage fields:

- *seqno (monotonic)*
- *ts (timestamp)*
- *iv*
- *ct (AES ciphertext)*

- *sig (RSA signature)*

Signature covers:

(seqno + ts + plaintext_msg)

This ensures:

- *integrity*
- *authenticity*
- *non-repudiation*

Server verifies:

- *signature matches client public key*
- *timestamp within allowed window (5 seconds)*
- *seqno exactly matches expected (replay protection)*

9. Replay Protection

Server maintains expected_seq.

If chat.seqno != expected_seq:

→ Reject message with SEQNO_MISMATCH.

This blocks:

- *replayed packets*
- *reordered packets*

- *tampering that changes seqno*

10. Transcript & SessionReceipt (Phase 7)

During chat:

server logs:

```
[  
  { seqno, ts, message, signature }  
]
```

On client sending Close:

- *server finalizes transcript JSON file*
- *computes transcript_hash = SHA256(file)*
- *signs hash with server private key*
- *sends Receipt(transcript_hash, signature)*

Client verifies signature using server certificate.

This gives:

- ✓ *Non-repudiation*
- ✓ *Forensic logging*
- ✓ *End-to-end verifiable communication history*

11. Database Schema (MySQL)


```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  username VARCHAR(255) UNIQUE NOT NULL,  
  salt BINARY(16) NOT NULL,  
  pwd_hash VARCHAR(64) NOT NULL  
);
```

12. Summary of All Security Guarantees

- ✓ Confidentiality — AES-128-CBC
- ✓ Authentication — mutual certificate verification
- ✓ Integrity — RSA signatures on each message
- ✓ Replay protection — strict seqno enforcement
- ✓ Forward secrecy — ephemeral DH & per-session DH
- ✓ Non-repudiation — signed transcript receipts
- ✓ Resist MITM — certificate-bound key derivation

This completes the report for Assignment 2: SecureChat.